

Presence Detection with Ultra-Wide Band Radar

Matteo Figini¹, Tiya Jreige¹, and Valeria De Gennaro¹

¹Politecnico di Milano

September 12, 2025

1 Introduction

Ultra-Wide Band (UWB) radar excels in high-resolution sensing, material penetration, and multipath robustness, making it ideal for precise target detection in diverse environments. However, raw UWB data is often obscured by static clutter, necessitating advanced signal processing for accurate interpretation.

This project aims to establish an efficient pipeline for UWB radar target detection and localisation, specifically for real-time embedded system applications. By integrating sophisticated signal processing with modern machine learning, we seek to unlock UWB radar's full potential for autonomous perception.

The main goals of this project are:

- **Robust Signal Preprocessing and Spatial Enhancement:** To effectively suppress clutter and accurately determine target range and azimuth angle through de-cluttering and digital beamforming, generating interpretable Range-Azimuth images.
- **Automated Object Detection and Localization:** To automatically identify and localize targets using a labelled dataset of beamformed images. This is achieved via the lightweight FOMO (Faster Objects, More Objects) deep learning model, trained and deployed through the Edge Impulse platform for efficient edge inference.

2 Design & Implementation

To develop this project, we followed the steps described in Repole's thesis [1].

2.1 Data collection

To collect the data, two different radars were used: a TrueSense SR250 and an Infineon PWMC radar. We applied the beamforming

technique and trained the object detection network using both types of radars.

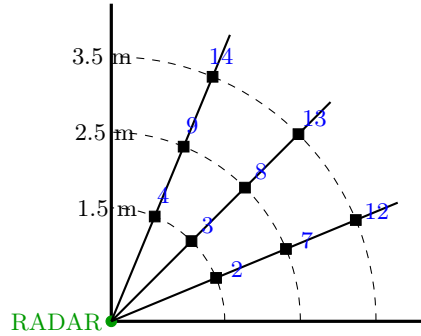


Figure 1: Data collection setup schema.

The data collection setup is depicted in Figure 1. Nine positions were considered from the radar location, respectively at a distance of 1.5, 2.5 and 3.5 metres, and considering angles of -22.5° , 0° and 22.5° .

A total of 90 samples were collected during the data collection, with one or more people (up to 4) who were standing in a specific location for 1 minute. Furthermore, in the dataset, there are also a couple of samples containing one person moving around different positions every 30 seconds.

For each sample and radar, three NumPy files are saved, each one containing the raw data for a single receiver. In the case of SR250, each file stores a NumPy matrix of size (1220, 120), with complex numbers as data type; the first dimension represents the number of time frames, while the second one represents the distance from the radar location as the number of range bins. Infineon raw data has a slightly more complex shape (`time_frames`, `num_antennas`, `num_chirps`, `samples_per_chirp`) and each element is a `float32` value, which must be pre-processed accordingly to [this section](#).

2.2 Preprocessing

In radar signal processing, decluttering removes static reflections (clutter) from the environment, thereby enhancing the visibility of moving targets. In this project, we explored two decluttering techniques: Simple Moving Average and Exponentially-Weighted Moving Average.

2.2.1 Simple Moving Average (SMA)

The SMA method estimates the background clutter by averaging a fixed number of the most recent radar frames. This technique operates as a **Finite Impulse Response (FIR) filter**, meaning the current background estimate is solely dependent on a finite, predefined window of past inputs. For each radar frame m and range bin k , the estimated background $\tilde{s}_{n,m}[k]$ is calculated by averaging the current frame $s_{n,m}[k]$ and the preceding $W - 1$ frames. The decluttered signal is then obtained by subtracting this average from the current frame:

$$\tilde{s}_{n,m}[k] = s_{n,m}[k] - \frac{1}{W} \sum_{i=m-W+1}^m (s_{n,i}[k])$$

where W is the window size. The output for the first $W - 1$ frames is typically set to zero as a full averaging window is not yet available.

2.2.2 Exponentially-Weighted Moving Average (EWMA)

The EWMA method provides a continuously adapting background estimate by assigning exponentially decreasing weights to older radar frames. This functions as an **Infinite Impulse Response (IIR) filter**, where the current output is influenced by all past inputs, albeit with a diminishing effect over time. The background estimate is initialised with the first radar frame, and each subsequent frame i is updated using a “forgetting factor” α :

$$\text{dec_base} = \text{dec_base} \cdot \alpha + \text{cirs}[i, :] \cdot (1 - \alpha)$$

A higher value of α (typically close to 1) results in slower adaptation to changes but offers greater stability for static clutter.

The result is a bidimensional matrix that represents the time-distance relationship, as shown in Figure 2. Note that only the distance of the radar is highlighted; indeed, different positions having the same distance from the radar are not distinguished.

2.2.3 Additional Preprocessing for Infineon radar

For the Infineon radar, additional preprocessing steps were applied to align with the spe-

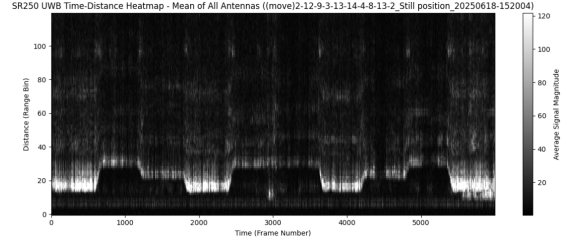


Figure 2: Time-distance plot with a person moving along positions 2-12-9-3-13-14-4-8-13-2 every 30 seconds.

cific data acquisition setup. The raw ADC data collected were normalized to the range $[-1, 1]$ to standardize the signal amplitude, preventing scaling issues. Then, for each frame, chirp and antenna constant biases were removed by subtracting the mean across all range bins. This ensures that the data is zero-centred across the range bins, and prevents DC components caused by static stutters or other biases from interfering with further processing and affecting real target values. Then again, for each frame, chirp and antenna, the Blackman-Harris windowing function was applied along the range bins. This ensures that strong reflection in one range bin does not affect other neighbouring bins, which improves the accuracy and result of the FFT results performed in later steps of the preprocessing pipeline, thus reducing leakage. The data were then zero-padded along the range bin dimension, which gives a finer range resolution in the frequency domain, basically increasing the FFT resolution.

Then the FFT is applied, and this step converts the data collected from each chirp into a range bin representation, where each frequency is associated with a specific target distance. Only positive frequencies are kept since they represent actual range distances, and their amplitude is doubled to preserve the signal power.

At the end of this processing pipeline, the range bin profiles are averaged across the chirp dimension, which reduces noise.

Then, beamforming is applied to the output of the preprocessing pipeline, which has the following shape (frames, range bins, antennas).

2.3 Digital Beamforming (DBF)

DBF is a signal processing technique applied to data received from an array of antennas to create a spatial filter, effectively steering the array’s sensitivity in a desired direction.

The core principle of DBF involves applying calculated complex weights to the individual signals from multiple receiver antennas, called ULA (Uniform Linear Array), before summing them.

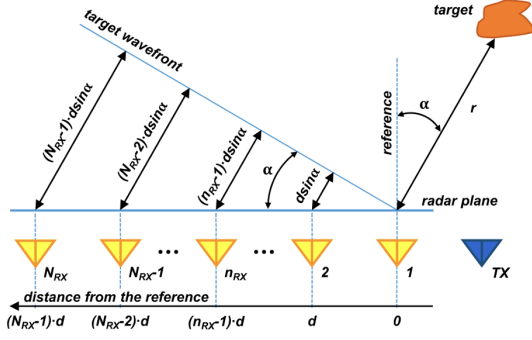


Figure 3: ULA of receiver antennas (taken from [1]).

These weights introduce specific phase shifts and amplitude adjustments. For a ULA of antennas, the phase shift for each element is designed to compensate for the differential propagation delay of a signal arriving from a particular angle. When signals from a target at the desired "beam-steering angle" are coherently summed, their phases align, leading to constructive interference and a strengthened signal. Conversely, signals from other directions will interfere destructively, resulting in their suppression.

In this project, we implemented DBF by:

- Utilizing data from two specific receiver antennas (RX0 and RX2, in the SR250 case) to form a ULA of 2 antennas.
- Calculating beamforming weights for a range of azimuth angles based on antenna spacing (δ_{RX}), wavelength ($\lambda = c/f$), and wavenumber ($k = 2\pi/\lambda$). Assuming a distance r and an angle α between the target and the first array antenna, the signal reflected from the target should travel an additional distance of $\Delta_r = (n_{RX} - 1) * \delta_{RX} * \sin(\alpha)$ to reach the n_{RX} -th antenna, corresponding to a phase difference of $\Delta_\phi = \frac{2\pi\delta_{RX}}{\lambda} * \sin(\alpha)$ among the signals received by adjacent antennas, where λ is the wavelength of the transmitted radar signal.
- The DBF weights are obtained as:

$$W(\Theta_i) = \exp[-j \frac{2\pi}{\lambda} \delta_{RX} (n_{RX} - 1) \sin(\Theta_i)] \quad (1)$$

- Applying these weights to the complex time-series data from the selected antennas via a vectorized matrix multiplication to produce a 3D (time, range, angle) beamformed output, as the dot product between the DBF weights and the antenna data, stacked together.

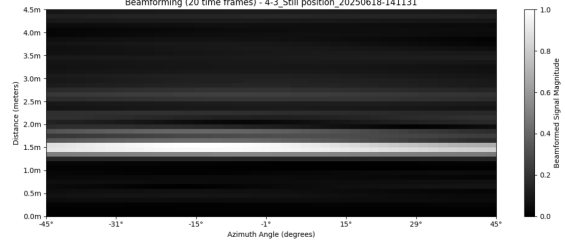


Figure 4: Digital Beamforming on positions 3 and 4, using SR250.

This process transforms the raw time-domain radar measurements into a spatial representation, enabling the pinpointing of the angular location of targets relative to the radar system.

2.4 Object Detection

Following DBF, the project aims to implement an object detection system to identify and localize targets within the radar scene automatically. This involves generating a specialised dataset of images from the beamformed radar data and then training a deep learning model for detection.

The foundation for the object detection task lies in the creation of a comprehensive image dataset derived from the 3D beamformed radar data (time, range, angle). The process systematically transforms the continuous radar streams into a sequence of 2D Range-Azimuth images suitable for deep learning.

For the **image generation process**, we followed these steps:

- The full 3D beamformed data, representing radar reflections across time, range bins, and azimuth angles, serve as the input.
- A **sliding time window** approach is employed. For each window (e.g., 40 frames, with a stride of 20 frames for non-overlapping windows), the complex data within that window are aggregated.
- **Averaging** is performed across the time frames within each window. This enhances the signal of persistent targets and reduces potential noise.
- The resulting 2D aggregated data is then **normalised** to a range $[0, 1]$ using min-max scaling, preparing it for visualization and model input for the SR250 radar, since this step was already performed for the Infineon radar.

- The 2D aggregated range-azimuth heatmaps are then treated as PNG images for the object detection task.

Moreover:

- For the SR250 radar, the normalized 2D Range-Azimuth maps are saved as grayscale PNG images. These images are rendered to a fixed resolution with axes turned off, ensuring a consistent input format for the object detection model.
- For the Infineon radar, the normalized 2D Range-Azimuth maps are saved both as RGB and grayscale PNG images. Crucially, these images are saved as a 96×96 image resolution with axes turned off, ensuring a consistent input format for the object detection model.

Bounding Box Labeling: For each generated image, the corresponding array of bounding boxes is saved in the *.labels* file. These bounding box annotations map to the known ground truth positions and are extracted from the original ‘.npy’ filenames (e.g., "12-14_Still position"), are converted into pixel coordinates (*x*, *y*, *bb_width*, *bb_height*) relative to the dimension of the input image to the object detection network (e.g. 96×96). A standard bounding box size is used around the centre of the projected ground truth location, providing the necessary labels for supervised learning.

2.4.1 Object Detection Model: FOMO Network

For the task of detecting objects within these generated radar images, a specialised deep learning model known as **FOMO (Faster Objects, More Objects)** is employed. FOMO is particularly well-suited for embedded and real-time object detection applications, especially when dealing with smaller objects and lower-resolution imagery, which aligns with the characteristics of the generated radar heatmaps.

FOMO operates by dividing the input image into a grid and predicting whether an object’s centroid falls into each grid cell. This approach is highly efficient for detecting multiple small objects without the overhead of traditional anchor box-based detectors. Its lightweight architecture makes it ideal for deployment on resource-constrained platforms, aligning with potential future applications of UWB radar for embedded sensing. The model is trained on the generated dataset of 2D Range-Azimuth images and their corresponding bounding box labels to learn the spatial patterns indicative of target presence and location.

In this project, FOMO models, which are available by exploiting MobileNetV2 0.1 and MobileNetV2 0.35 as a convolutional backbone, were trained and deployed using **Edge Impulse**, which is a leading development platform for machine learning on edge devices. It provides a comprehensive set of tools that streamline the entire machine learning workflow, from raw data collection and labelling, preprocessing, to model training, optimisation, and deployment on diverse hardware. A detailed explanation of the model can be found online at [2].

3 Results

To train the object detection model, we experimented with several model configurations and varying dataset sizes. For both datasets, we performed object detection either by considering 9 distinct positions and using them as classes, or by considering a single class "person"/"human". Training using 9 different positions is useful:

- To check which positions are hard to detect;
- To guide data collection and check which positions need more samples (unbalanced data);
- In case we know a priori the positions where the target object will be located.

On the other hand, training using a single label is useful:

- To work with new positions not seen during training;
- To learn a unified person representation.

3.1 SR250 Radar Results

The results shown here refer to a version in which, from each sampling of 1200 time frames (after the removal of the first 20 "noisy" samples), windows of size 40 time frames and a step size of 20 are extracted, i.e. the first window extracted for beamforming is in the range [0, 40] time frames, the second window is in the range [20, 60], and so on. The dataset consists of 5790 images of size 155×154, split between the training set and the test set in an 80%/20% proportion. The training set is further split in an 85%/15% proportion between the training set and validation set.

	BACKGROUND	PERSON
BACKGROUND	100.0%	0.0%
PERSON	13.9%	86.1%
F1 SCORE	1.00	0.63

Figure 5: Confusion Matrix on the Validation Set for SR250 data, using MobileNetV2 0.35 (float32).

	BACKGROUND	PERSON
BACKGROUND	100.0%	0.0%
PERSON	19.9%	80.1%
F1 SCORE	1.00	0.60

Figure 6: Confusion Matrix on the Validation Set for SR250 data, using MobileNetV2 0.35 (int8).

The confusion matrix on the validation set, reported in Figure 5 for the unoptimized version and in Figure 6 for the corresponding quantized version, highlights the predicted targets compared against the actual targets in the validation set.

From the experimental results, we observed that considering one unique class for the problem leads to higher precision and higher recall on the validation/test set. Differences among the models are highlighted in Table 1.

3.2 Infineon Radar Results

	BACKGRO	12	13	14	2	3	4	7	8	9
BACKGRO	100.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
12	0.0%	100.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
13	0.0%	0.0%	100.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
14	0.0%	0.0%	0.0%	100.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
2	0.0%	0.0%	0.0%	0.0%	100.0%	0.0%	0.0%	0.0%	0.0%	0.0%
3	0.0%	0.0%	0.0%	0.0%	0.0%	100.0%	0.0%	0.0%	0.0%	0.0%
4	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	100.0%	0.0%	0.0%	0.0%
7	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	100.0%	0.0%	0.0%
8	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	100.0%	0.0%
9	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	100.0%
F1 SCORE	1.00	0.65	0.32	0.49	0.77	0.54	0.17	0.68	0.31	0.23

Figure 7: Classification of grayscale 96×96 images with 9 labels.

	BACKGRO	12	13	14	2	3	4	7	8	9
BACKGRO	100.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
12	0.0%	100.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
13	0.0%	0.0%	100.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
14	0.0%	0.0%	0.0%	100.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
2	0.0%	0.0%	0.0%	0.0%	100.0%	0.0%	0.0%	0.0%	0.0%	0.0%
3	0.0%	0.0%	0.0%	0.0%	0.0%	100.0%	0.0%	0.0%	0.0%	0.0%
4	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	100.0%	0.0%	0.0%	0.0%
7	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	100.0%	0.0%	0.0%
8	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	100.0%	0.0%
9	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	100.0%
F1 SCORE	1.00	0.64	0.52	0.59	0.80	0.54	0.53	0.77	0.66	0.46

Figure 8: Classification of RGB 96×96 images with 9 labels.

	BACKGROUND	HUMAN
BACKGROUND	100.0%	0.0%
HUMAN	29.5%	70.5%
F1 SCORE	1.00	0.71

Figure 9: Classification of 96×96 RGB images with 1 label.

	BACKGROUND	HUMAN
BACKGROUND	100.0%	0.0%
HUMAN	35.3%	64.7%
F1 SCORE	1.00	0.71

Figure 10: Classification of 64×64 RGB images with 1 label.

The results shown here refer to a version in which, from each sampling of 1200 time frames, windows of size 20 time frames and a step size of 20 are extracted, i.e. the first window extracted

for beamforming is in the range [0, 20] time frames, the second window is in the range [20, 40], and so on. The dataset consists of 5,368 images of size 96×96 (here the moving samples are not considered), split between the training set and the test set in an 80%/20% proportion. The training set is further divided into an 85%/15% proportion between the training set and the validation set.

Different experiments were conducted using the Infineon radar, considering training both with 9 different classes and with 1 class. All of these models were also trained on RGB and grayscale images of sizes 96×96 or 64×64. The results can be observed in Table 3.

In Figure 7, and Figure 8, we can see that the network may find issues in identifying targets at specific positions 4, 9 and 13.

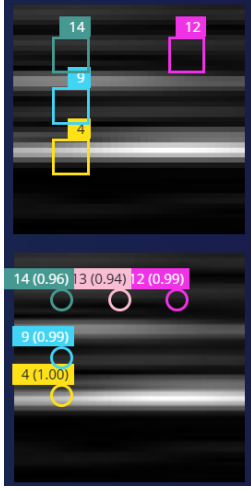
Furthermore, there are no mismatches between the classes found in the confidence matrices in Figures 7, 8, 9 and 10, whereas a mismatch always involves a class and the background class. This happens because the positions inside the images, and therefore the bounding boxes, are always in the same place, since they refer to the same specific position in the collection scenario. This is also the reason for having high recall and low precision, and suggests that the number of false positives detected by the model is constantly higher than the number of false negatives.

4 Discussion

As mentioned before, during the project, we performed several trials both in the beamforming part and in the object detection part.

The first consideration is that the results of the DBF, employing either the SMA or the EWMA, or both methods combined, are quite similar and don't change the results in the object detection part significantly. For the SR250 radar, we decided to use both decluttering techniques, applying first the SMA, then the EWMA.

Another consideration, for the SR250 radar, is that the results of beamforming we obtained may not clearly distinguish the "blobs" around the correct position of the target sometimes. This could be caused by physical limitations of the specific radar we focused on, or issues in the data collection procedure. Exploring these limitations could be the key to improving the object detection step. In Figures 11a and 11b, it is possible to see an example of this specific behaviour when considering the 9-classes problem.



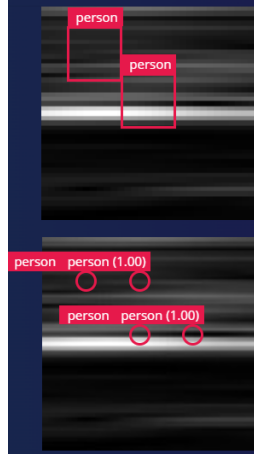
(a) Detection with F1-Score: 88.9%.



(b) Detection with F1-Score: 50.0%.



(c) Detection with F1-Score: 100.00%.



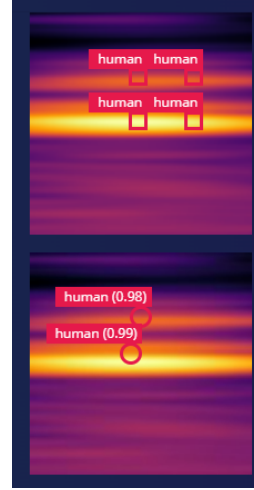
(d) Detection with F1-Score: 66.7%.

Figure 11: Predictions of targets for 9-classes and 1-class models of SR250 radar.

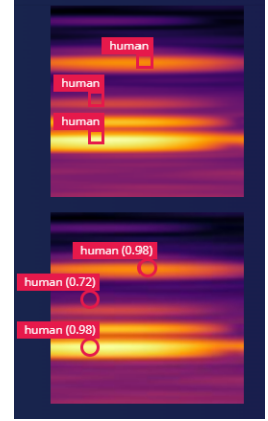
Furthermore, we can appreciate that the object detection shows that even a model like FOMO can solve the task of object detection using very constrained hardware, such as a microcontroller unit.

The Infineon radar performed well in identifying targets in different range bins. However, there is a problem with the locations located in the same range bin, due to the fact that they have elongated blobs that could overlap with one another. This limitation arises from the fact of using 2 receiving antennas for beamforming, which reduces the angular resolution, making the result less sharp, and having less accurate separation between different targets at the same range bin. This also leads to targets at the same range bin to be confused as one, as shown in Figure 12a.

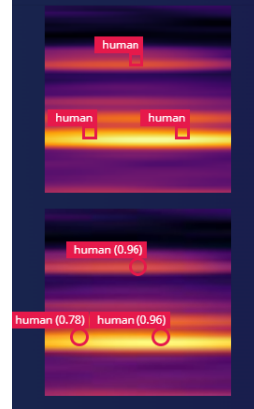
It is also depicted in the images above that the



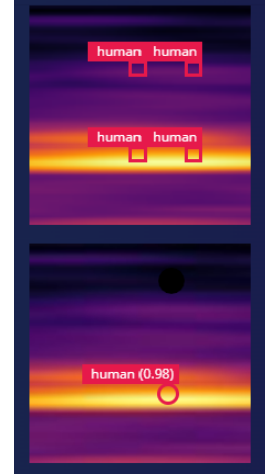
(a) Targets at positions 7-8-13-14, 96×96 images.



(b) Targets at positions 9-13-14, 96×96 images.



(c) Targets at positions 2-4-13, 64×64 images.



(d) Targets at positions 2-3-12-13, 64×64 images.

Figure 12: Predictions of targets for different positions and image sizes for Infineon radar.

brightness of the blob is reduced whenever there is a target detected in the same position but at a closer range bin from the radar. For instance, if there is a person between the radar antennas and another person further away, the latter is barely visible in the images obtained through beamforming, making the person further away less visible and not detected, as shown in Figure 12d.

4.1 Comparison of Results

4.1.1 SR250 Radar

In this section, we will discuss the results of the different experiments that we have done. Table 1 below shows the difference between the metrics resulting from training using **MobileNetV2 0.35** and the ones resulting from training using **MobileNetV2 0.1** using the data collected

through the SR250 radar.

The quantised version of the model can be uploaded to a microcontroller, considering the following requirements in case of **MobileNetV2 0.35**:

- EON Compiler: **Latency:** 811 ms / **RAM:** 239.4 KB / **Flash:** 70.7 KB
- TensorFlow Lite: **Latency:** 811 ms / **RAM:** 283.1 KB / **Flash:** 110.3 KB

And the following ones in case of **MobileNetV2 0.1**:

- EON Compiler: **Latency:** 751 ms / **RAM:** 235.5 KB / **Flash:** 58.4 KB
- TensorFlow Lite: **Latency:** 751 ms / **RAM:** 280.7 KB / **Flash:** 94.3 KB

4.1.2 Looking for highly-relevant blobs

A recurring pattern that emerges from the analysis of images obtained following beamforming of the SR250 radar shows that, if there is a person between the radar antennas and another person further away, the latter is barely visible in the images obtained through beamforming, making the person further away less visible. This behaviour is clearly visible via an experiment we ran to understand whether brighter spots are caught in the positions inside the image where the target is expected to be identified. Normalising the values of the range-angle matrix between 0 (corresponding to black) and 1 (corresponding to white, which identifies a person), we looked for the best blobs among all the time frames, considering windows of size 40 time frames and a step size of 10 time frames. The results are plotted in Figure 13, which is the representation of a matrix with dimensions (9, 1200): the first dimension is the number of positions, while the second one is the time frames. Here, it is possible to note how positions further away from the radar are more difficult to detect. The criteria for the blob detections are to have a value greater than or equal to 0.5 in a region corresponding to a neighbourhood of the expected target position.

4.1.3 Infineon radar

For the Infineon radar, we have performed several experiments, with grayscale images, RGB images and different sizes of the bounding boxes. Table 3 shows the difference between the metrics resulting from training using **MobileNetV2 0.35** and the ones resulting from training using **MobileNetV2 0.1** for the different experiments performed.

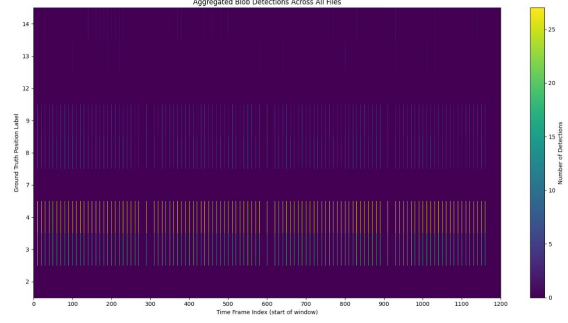


Figure 13: Results of the best blob detection on SR250 data.

The quantized version of the model can be uploaded to a microcontroller, considering the following requirements in case of **MobileNetV2 0.35**:

- Grayscale Images:
 - 64×64 images: **Latency:** 359 ms / **RAM:** 117.9KB / **Flash:** 70.8KB
- RGB Images:
 - 64×64 images: **Latency:** 367 ms / **RAM:** 118.0KB / **Flash:** 71.0K
 - 96×96 images: **Latency:** 786 ms / **RAM:** 239.5KB / **Flash:** 71.0KB

And the following ones in case of **MobileNetV2 0.1**:

- Grayscale:
 - 96×96 images: **Latency:** 740 ms / **RAM:** 235.5KB / **Flash:** 58.5KB
- RGB Images:
 - 64×64 images: **Latency:** 344 ms / **RAM:** 114.1KB / **Flash:** 58.6KB

5 Conclusions

This project proved to be very interesting for us as it allowed us to explore the challenges of digital beamforming and object detection in depth. Working on these aspects not only improved our technical understanding but also gave us a clearer view of the potential of UWB radar technology and the deployment of object detection models on resource-constrained devices such as microcontrollers.

Future improvements could consider further processing in the context of digital beamforming, while also experimenting with additional object detection models. Moreover, since this project has numerous practical applications, it is possible to put its pipeline into a real-world context.

Number of classes	FOMO Model	Version	Precision	Recall	F1-Score
9 classes & background	MobileNetV2 0.35	Unoptimized (float32)	42.3%	83.5%	56.2%
		Quantized (int8)	42.0%	73.6%	53.5%
	MobileNetV2 0.1	Unoptimized (float32)	40.3%	86.0%	54.9%
		Quantized (int8)	41.0%	83.9%	55.1%
1 class & background	MobileNetV2 0.35	Unoptimized (float32)	49.2%	86.1%	62.6%
		Quantized (int8)	48.3%	80.1%	60.2%
	MobileNetV2 0.1	Unoptimized (float32)	45.5%	87.0%	59.8%
		Quantized (int8)	47.0%	83.1%	60.1%

Table 1: SR250 metrics values for FOMO MobileNetV2 0.1 & 0.35.

Image Size	Experiment	FOMO Model	Version	Precision	Recall	F1-Score
96×96	RGB	MobileNetV2 0.35	Unoptimized (float32)	78.0%	78.0%	78.0%
			Quantized (int8)	78.0%	68.0%	73.0%
	Grayscale	MobileNetV2 0.1	Unoptimized (float32)	71.0%	70.0%	70.0%
			Quantized (int8)	71.0%	70.0%	71.0%
64×64	RGB	MobileNetV2 0.35	Unoptimized (float32)	84.0%	67.0%	75.0%
			Quantized (int8)	75.0%	57.0%	65.0%
	Grayscale	MobileNetV2 0.1	Unoptimized (float32)	77.0%	62.0%	69.0%
			Quantized (int8)	79.0%	64.0%	71.0%
	Grayscale	MobileNetV2 0.35	Unoptimized (float32)	80.0%	68.0%	74.0%
			Quantized (int8)	82.0%	62.0%	71.0%

Table 2: Infineon metrics values for FOMO MobileNetV2 0.1 & 0.35 for 1-class object detection.

Experiment	FOMO Model	Optimization	Precision	Recall	F1-Score
RGB	MobileNetV2 0.35	Float32 (Unoptimized)	52.0%	77.0%	62.0%
		Int8 (Quantized)	48.0%	82.0%	61.0%
	MobileNetV2 0.1	Float32 (Unoptimized)	50.0%	79.0%	61.0%
		Int8 (Quantized)	48.0%	78.0%	59.0%
Grayscale	MobileNetV2 0.1	Float32 (Unoptimized)	56.0%	79.0%	66.0%
		Int8 (Quantized)	57.0%	42.0%	48.0%

Table 3: Infineon metrics values for FOMO MobileNetV2 0.1 & 0.35 for 9-classes object detection (image size 96×96).

5.1 The ethical perspective

Unlike cameras, UWB radar does not capture visual identity (faces, clothing, etc.), which makes it less intrusive and more privacy-preserving. However, it detects presence, movement, and potentially gestures or breathing, so people inside the radar area must be aware and consent to being tracked. Moreover, transparency about the data collection, processing and storage of the data is required; indeed, a potential issue in our project is given by the usage of an external platform (Edge Impulse) to store the acquired data and train the model.

If the object detection algorithm has not been trained on diverse body types (e.g. children, tall people, wheelchair users and so on), postures,

or mobility aids, it might systematically miss or misclassify certain targets; thus, fairness in detection must be ensured to avoid exclusion or unintended discrimination.

5.2 The market perspective

UWB radar offers privacy-friendlier sensing, works in darkness or through obstacles, and can detect subtle movements (breathing, falls); however, they come with a lower resolution than vision systems, and may face adoption resistance due to novelty.

Possible applications are:

- **Healthcare and Elderly Care** for non-intrusive fall detection or vital signs moni-

toring.

- **Smart Homes and Buildings:** e.g. we can adapt the level of lights or air conditioning inside a room depending on the presence of one or more people.
- **Intrusion Detection** without cameras.
- **Child presence detection** inside vehicles based on the presence or the breathing acts, as done in [3] and [4].

Acknowledgements

We thank Prof. Pavan for the patience and the support he gave us during the execution of the project.

References

- [1] G. Repole, “Tinymml-enabled human indoor localization with uwb-radar imaging”, 2023.
- [2] *FOMO - Edge Impulse*, <https://docs.edgeimpulse.com/studio/projects/learning-blocks/blocks/object-detection/fomo>, [Online].
- [3] M. Pavan, A. Caltabiano, M. Roveri, *et al.*, “On-device subject recognition in uwb-radar data with tiny machine learning”, in *CEUR Workshop Proceedings*, vol. 3252, 2022, pp. 1–8.
- [4] M. Pavan, A. Caltabiano, and M. Roveri, “Tinymml for uwb-radar based presence detection”, in *2022 International Joint Conference on Neural Networks (IJCNN)*, IEEE, 2022, pp. 1–8.